

05 - Development Workflow & Session Guide

Project: Dragon Merchant Guild (*Working Title*)

Document Version: 1.0

Status: Active

1. Purpose

This document defines **how development sessions are conducted**.

Its purpose is to eliminate decision fatigue before, during, and after studying.

Instead of asking:

"What should I do today?"

the developer simply follows the workflow.

2. Guiding Principle

Every study session should produce **one meaningful improvement**.

A meaningful improvement is not measured by the amount of code written, but by the amount of understanding gained.

3. Standard Study Session

Each study session follows the same overall structure.

Prepare

↓

Study

↓

Plan

↓

Build
↓
Test
↓
Review
↓
Reflect
↓
Journal

The duration of each step is intentionally flexible.

Understanding is more important than speed.

4. Phase 1 — Prepare

Before opening the editor:

- Review the current curriculum topic.
- Read the relevant section of the Curriculum Integration Guide.
- Review any notes from the previous session.
- Decide on today's single objective.

Example

Today's topic:

Methods & the C-corner

Today's objective:

Implement the first contract management actions.

5. Phase 2 — Study

Study until the concepts make sense.

Do not attempt to memorize syntax.

Instead, focus on understanding:

- what problem the feature solves,

- why it exists,
- and when it should be used.

If necessary:

- make handwritten notes,
 - draw diagrams,
 - compare with C,
 - ask conceptual questions.
-

6. Phase 3 — Plan

Before writing code, spend a few minutes planning.

Ask questions such as:

- What feature am I building?
- Which concepts does it use?
- What objects already exist?
- Does this feature belong somewhere else?
- Can I keep this simple?

The goal is not to design everything.

The goal is to avoid coding blindly.

7. Phase 4 — Build

Open the editor.

Begin implementing.

During this phase:

- AI remains closed.
- Documentation may remain open.
- Experiment freely.
- Make mistakes.
- Refactor if necessary.

If blocked:

Attempt several approaches before asking for help.

8. Phase 5 — Test

After implementation:

Run the game.

Play with the new feature.

Intentionally try incorrect inputs.

Attempt to break your own implementation.

Questions to ask:

- Does it behave correctly?
- Does it handle invalid input?
- Does it crash?
- Does it make sense to a player?

9. Phase 6 — Review

Only after making an honest attempt should AI be consulted.

Recommended review prompts include:

- Review this feature.
- Challenge my implementation.
- Ask questions about my reasoning.
- Suggest possible refactors.
- Point out code smells.
- Identify edge cases.

Avoid asking AI to rewrite the feature.

10. Phase 7 — Reflect

After every session, answer these questions:

What did I learn?

What confused me?

What finally "clicked"?

What would I do differently next time?

Which future topic do I think will improve this feature?

Reflection transforms experience into understanding.

11. Phase 8 — Journal

Record a brief development log.

Example:

Date:

2026-08-01

Topic:

Methods & the C-corner

Feature:

Implemented contract acceptance and dragon assignment.

Biggest Lesson:

`out` parameters require assignment before returning.

Next Goal:

Improve contract lookup using Collections.

12. Session Rules

During development:

- ✓ Build one feature at a time.
- ✓ Keep the game playable.
- ✓ Prefer simple solutions.
- ✓ Refactor when understanding improves.
- ✓ Commit regularly.

Avoid:

- ✗ Implementing unrelated features.
 - ✗ Optimizing prematurely.
 - ✗ Chasing perfect architecture.
 - ✗ Comparing progress to others.
-

13. Definition of "Done"

A study session is considered complete when:

- The concept is understood.
- The feature works.
- The code is readable.
- The game still runs.
- Reflection has been completed.

Not when:

- Every possible improvement has been made.

Perfection is not required.

Progress is.

14. Weekly Review

At the end of each week:

Review:

- completed topics,
- implemented features,
- recurring mistakes,
- refactoring opportunities,
- remaining questions.

Ask:

"If I restarted this project today, what would I do differently?"

Do not immediately perform the rewrite.

Instead, record the insight for future refactoring.

15. Refactoring Rule

Refactoring is encouraged only when one of the following is true:

- New knowledge makes the code significantly simpler.
- Duplication has become obvious.
- The code is difficult to understand.
- A bug repeatedly appears.
- A curriculum chapter specifically introduces a better solution.

Avoid refactoring simply because a different design "might" be better.

16. Success Criteria

A successful development session leaves the developer with:

- greater confidence,
- deeper understanding,
- a working game,
- and one fewer unknown than before.

That is sufficient.

Personal Reminder

Small improvements, repeated consistently, build both the game and the developer.